

Rainfall Patch Generation – Full Specification

1. Input Data

Files: EURADCLIM ODIM HDF5 rainfall maps.

Directory: backend/data/raw/

Pattern: RAD_OPERA_HOURLY_RAINFALL_ACCUMULATION_YYYYMMDDHHMM.h5

Extracted fields include rainfall matrix, timestamp, and geolocation corner coordinates.

2. Detection Parameters

threshold_mm: Rain threshold for mask creation.

avg_window_y/x: Pixel smoothing window.

min/max_width_km, min/max_height_km: Patch size constraints.

max_files: Number of recent files to auto-scan.

files: Optional explicit list of .h5 files.

3. Patch Detection Algorithm

1. Load rainfall array and clean missing values.
2. Apply optional smoothing if avg_window > 1.
3. Threshold rainfall to form binary mask.
4. Perform connected-component labeling.
5. Compute bounding boxes for each component.
6. Convert bounding box pixel size → km.
7. Filter by min/max km constraints.
8. Compute rainfall stats (mean, max, area km²).
9. Compute center lat/lon using bilinear interpolation grid.
10. Determine nearest city.
11. Assign patch ID based on file name + index.

4. Patch Output Structure

Each patch includes:

- id, source_file, timestamp
- y_min, y_max, x_min, x_max
- mean_rainfall, max_rainfall
- width_km, height_km, area_km2
- center_lat, center_lon
- nearest_city

Raw cropped rainfall array stored internally for image generation.

5. Patch Image Generation

- Uses cropped rainfall array (`_data`).
- Rendered via `matplotlib imshow()`.
- Colorbar = rainfall mm.
- Includes axis ticks and pixel coordinates.
- Delivered as PNG through `/patch_image/{id}`.

6. Geographic Metadata

- Converts km width/height to degrees using 111 km per degree.
- Adjusts longitude scaling by $\cos(\text{latitude})$.
- Computes patch bounding box in lat/lon.
- Computes larger map bounding box using `zoom_factor` (default 3×).
- Used by frontend OSM map to draw rectangle and set view bounds.